# Bi-Weekly Progress Report

SQL to MongoDB Query Transpiler

**Group 7**

G. Sowmya Sri CS23B019

M. Pranathi CS23B030

P. Niharika CS23B035

---

## 1. Work Done So far

Prior to the current phase, the transpiler has a robust, end-to-end pipeline capable of parsing and translating standard SQL SELECT queries into executable MongoDB syntax. The previously implemented components include:

- **Lexical Analysis:** A PLY-based tokenizer that accurately identifies SQL keywords, identifiers, operators, and literals.
- **Syntax Analysis & AST Generation:** A grammar-driven parser that validates the query structure and constructs an Abstract Syntax Tree (AST) using modular nodes (SelectQuery, LogicalCondition, Comparison).
- **Semantic Analysis:** A validation layer featuring custom schema loading to strictly enforce table/column existence, resolve data-type compatibility, and ensure logical correctness.
- **Code Generation:** A translation engine that successfully maps validated AST structures into standard MongoDB db.collection.find() queries, bridging operators and logical conditions for the basic query pipeline.
- **Custom Schema Loader**

---

## 2.Work Planning to Demonstrate

In this stage, the following features have been successfully implemented:

- **Aggregate Functions**: Support for COUNT, SUM, AVG, MIN, and MAX.
- **GROUP BY Clause**: Grouping records based on one or more columns.
- **HAVING Clause**: Post-aggregation filtering.

- **ORDER BY Clause**: Sorting result sets in ascending (ASC) or descending (DESC) order.
- **LIMIT Clause**: Restricting the total number of returned documents.
- **Combined Pipeline**: Seamless integration of GROUP BY, HAVING, ORDER BY, and LIMIT features to handle complex queries simultaneously.
- **Additional Enhancements**: Support for **multiple GROUP BY columns**, allowing results to be aggregated over compound criteria.

# Implementation Details

The transpiler's core modules were carefully updated to accommodate the new grammar rules and AST structures.

## 2.1 AST Changes

To parse and represent the new query components, the parser logic was expanded to encompass the following new AST nodes and attributes:

- **Aggregate Node**: Represents aggregate functions within the abstract syntax tree. It stores the func (e.g., COUNT, SUM) and the corresponding column argument.
- **OrderByItem Node**: Encapsulates an individual column and its sorting direction (ASC or DESC).
- **SelectQuery Enhancements**: The base SelectQuery node was extended with optional attributes:
    - group_by: A list of strings representing the columns to group by.
    - having: An ASTNode (like LogicalCondition or Comparison) applied for aggregating filtering.
    - order_by: A list of OrderByItem instances representing the sorting sequence.
    - limit: An integer denoting the cap on the results returned.

## 2.2 Semantic Analysis Updates

The semantic rules for query validation were fortified:

1 **GROUP BY Enforcement**: Any column listed in the SELECT clause that is not enclosed within an aggregate function must be explicitly stated in the GROUP BY clause.

2 **HAVING Pre-requisite**: A semantic validation explicitly checks that a HAVING clause is strictly accompanied by a GROUP BY clause. If left missing, the transpiler accurately halts by throwing a SemanticError.

3 **Aggregate Scoping**: Function validations confirm that columns wrapped inside aggregate handlers (except COUNT(*)) exist within the established sub-schema parameters.

### 2.3 MongoDB Aggregation Pipeline

Due to the constraints of db.collection.find(), queries involving aggregates or GROUP BY clauses necessitate the usage of MongoDB's Aggregation Framework. The transpiler was modified to intelligently swap out standard .find() for db.collection.aggregate([...]). The query conditions dynamically sequence into specialized stages such as $match, $group, $sort, and $limit.

## Mapping Rules (SQL → MongoDB)

The translation behavior constructs the MongoDB query stage-by-stage based on SQL clauses:

| SQL Construct | MongoDB Equivalent | Notes |
|---|---|---|
| GROUP BY col | $group: { _id: "$col" } | For multiple columns, maps to { _id: { col1: "$col1", col2: "$col2" } } |
| COUNT(*) | { "count": { $sum: 1 } } | Accurately counts matching documents per group. |
| COUNT(col) | { "count_col": { $sum: { $cond: [...] } } } | Validates non-null values for numeric aggregation. |
| SUM(col) | { "sum_col": { $sum: "$col" } } | Accumulates numeric values identically. |
| AVG(col) | { "avg_col": { $avg: "$col" } } | Averages values inside grouped keys. |
| MIN(col) | { "min_col": { $min: "$col" } } | Extracts literal minimum per field. |
| MAX(col) | { "max_col": { $max: "$col" } } | Extracts literal maximum per field. |
| HAVING cond | $match (Post-$group) | Implemented as a subsequent $match stage traversing aggregates. |
| ORDER BY | $sort | Values set to 1 for ASC, -1 for DESC. |
| LIMIT n | $limit | Simply caps array lengths. |

## Test Cases with Expected Outputs

### 🔹 TC: Basic Aggregate

**Input SQL**

```
SELECT COUNT(*) FROM users;
```

**MongoDB Output:**

```
db.users.aggregate([
  { '$group': { '_id': None, 'count': { '$sum': 1 } } }
])
```

### 🔹 TC: GROUP BY

**Input SQL**

```
SELECT city, COUNT(*) FROM users GROUP BY city;
```

**MongoDB Output:**

```
db.users.aggregate([
  { '$group': { '_id': '$city', 'count': { '$sum': 1 } } }
])
```

### 🔹 TC: SUM

**Input SQL**

```
SELECT city, SUM(age) FROM users GROUP BY city;
```

**MongoDB Output:**

```
db.users.aggregate([
  { '$group': { '_id': '$city', 'sum_age': { '$sum': '$age' } } }
])
```

### 🔹 TC: AVG

**Input SQL**

```
SELECT city, AVG(age) FROM users GROUP BY city;
```

**MongoDB Output:**

```
db.users.aggregate([
  { '$group': { '_id': '$city', 'avg_age': { '$avg': '$age' } } }
])
```

### 🔹 TC: HAVING

**Input SQL**

```
SELECT city, COUNT(*) FROM users GROUP BY city HAVING COUNT(*) > 2;
```

**MongoDB Output:**

```
db.users.aggregate([
  { '$group': { '_id': '$city', 'count': { '$sum': 1 } } },
  { '$match': { 'count': { '$gt': 2 } } }
])
```

### 🔹 TC: ORDER BY

**Input SQL**

```
SELECT name, age FROM users ORDER BY age DESC;
```

**MongoDB Output:**

```
db.users.find({}, { name: 1, age: 1 }).sort({ age: -1 })
```

(Observation: Query safely uses .find().sort() without entering the heavier .aggregate() pipeline)

### 🔹 TC: LIMIT

**Input SQL**

```
SELECT name FROM users LIMIT 5;
```

**MongoDB Output:**

```
db.users.find({}, { name: 1 }).limit(5)
```

### 🔹 TC: GROUP BY + ORDER BY

**Input SQL**

```
SELECT city, COUNT(*) FROM users GROUP BY city ORDER BY COUNT(*)
DESC;
```

**MongoDB Output:**

```
db.users.aggregate([
  { '$group': { '_id': '$city', 'count': { '$sum': 1 } } },
  { '$sort': { 'count': -1 } }
])
```

### 🔹 TC: GROUP BY + LIMIT

**Input SQL**

```
SELECT city, SUM(age) FROM users GROUP BY city LIMIT 5;
```

**MongoDB Output:**

```
db.users.aggregate([
  { '$group': { '_id': '$city', 'sum_age': { '$sum': '$age' } } },
  { '$limit': 5 }
])
```

### 🔹 TC: ORDER BY + LIMIT

**Input SQL**

```
SELECT name, age FROM users ORDER BY age DESC LIMIT 3;
```

**MongoDB Output:**

```
db.users.find({}, { 'name': 1, 'age': 1 }).sort({ 'age':
-1 }).limit(3)
```

### 🔹 TC: GROUP BY + HAVING

**Input SQL**

```
SELECT city, SUM(age) FROM users GROUP BY city HAVING SUM(age) >
100;
```

**MongoDB Output:**

```
db.users.aggregate([
  { '$group': { '_id': '$city', 'sum_age': { '$sum': '$age' } } },
  { '$match': { 'sum_age': { '$gt': 100 } } }
])
```

### 🔹 TC: HAVING + LIMIT

**Input SQL**

```
SELECT city, COUNT(*) FROM users GROUP BY city HAVING COUNT(*) > 5
LIMIT 10;
```

**MongoDB Output:**

```
db.users.aggregate([
  { '$group': { '_id': '$city', 'count': { '$sum': 1 } } },
  { '$match': { 'count': { '$gt': 5 } } },
  { '$limit': 10 }

])
```

])

### 🔹 TC: GROUP BY + ORDER BY + LIMIT

**Input SQL**

```
SELECT city, COUNT(*) FROM users GROUP BY city ORDER BY COUNT(*)
DESC LIMIT 3;
```

**MongoDB Output:**

```
db.users.aggregate([
  { '$group': { '_id': '$city', 'count': { '$sum': 1 } } },
  { '$sort': { 'count': -1 } },
  { '$limit': 3 }
])
```

### 🔹 TC: GROUP BY + HAVING + ORDER BY

**Input SQL**

```
SELECT city, SUM(age) FROM users GROUP BY city HAVING SUM(age) > 100
ORDER BY SUM(age) DESC;
```

**MongoDB Output:**

```
db.users.aggregate([
  { '$group': { '_id': '$city', 'sum_age': { '$sum': '$age' } } },
  { '$match': { 'sum_age': { '$gt': 100 } } },
  { '$sort': { 'sum_age': -1 } }
])
```

### 🔹 TC: Multiple GROUP BY Addition

**Input SQL**

```
SELECT city, role, COUNT(*) FROM users GROUP BY city, role;
```

**MongoDB Output:**

```
db.users.aggregate([
  { '$group': { '_id': { 'city': '$city', 'role': '$role' }, 'count':
 { '$sum': 1 } } }
])
```

### 🔹 TC: Edge Cases

**Input SQL**

```
SELECT city, COUNT(*) FROM users GROUP BY invalid_col;
```

**Output:** SemanticError (Column 'invalid_col' does not exist in table 'users')

**Input SQL**

```
SELECT city FROM users HAVING COUNT(*) > 5;
```

**Output:** SemanticError (HAVING clause requires GROUP BY)

---

# 3. Deviations

There are no deviations from the original project plan. All components described in the previous demonstration' s planned work have been implemented on schedule

# 4. Planned Work for Next Demonstration

The next phase will focus on integrating the transpiler with a real MongoDB instance to execute generated queries and display results for validation. A simple user interface will be developed for interactive demonstration of all pipeline stages. Additionally, support for more SQL clauses such as LIMIT with OFFSET, IN, and LIKE will be added, along with enhancements like multi-column GROUP BY and basic query optimization to improve efficiency and usability.